DBMS를 활용한 게시판 구축

20203067 서현우

database.js

mariadb를 사용하였으며 이는 mysql과 호환되므로 mysql 데이터베이스와의 연결을 설정하고 초기화 하였습니다. 다음은 테이블 설정입니다.

board(게시글)

#	이름	데이터 유형	길이/설정	부호 없음	NULL 허용	0으로...	기본값
1	**postid**	**INT**	**11**	☐	☐	☐	**AUTO_INCREME...**
2	author	VARCHAR	50	☐	☐	☐	기본값 없음
3	title	VARCHAR	255	☐	☐	☐	기본값 없음
4	body	TEXT		☐	☐	☐	기본값 없음
5	regdate	TIMESTAMP		☐	☑	☐	current_timestam...
6	viewcnt	INT	11	☐	☑	☐	'0'
7	likes	INT	11	☐	☑	☐	'0'

user테이블의 username을 외래키로 사용합니다.

reply(댓글)

#	이름	데이터 유형	길이/설정	부호 없음	NULL 허용	0으로...	기본값
1	**commentid**	**INT**	**11**	☐	☐	☐	**AUTO_INCREME...**
2	postid	INT	11	☐	☐	☐	기본값 없음
3	commentwriter	VARCHAR	50	☐	☐	☐	기본값 없음
4	commentcont...	TEXT		☐	☐	☐	기본값 없음
5	commentdate	TIMESTAMP		☐	☑	☐	current_timestam...

board의 postid와 user의 username을 외래키로 사용합니다.

user(사용자)

#	이름	데이터 유형	길이/설정	부호 없음	NULL 허용	0으로...	기본값
1	**username**	**VARCHAR**	**50**	☐	☐	☐	**기본값 없음**
2	password	VARCHAR	255	☐	☐	☐	기본값 없음
3	name	VARCHAR	50	☐	☐	☐	기본값 없음
4	email	VARCHAR	100	☐	☐	☐	기본값 없음
5	emaildomain	VARCHAR	50	☐	☐	☐	기본값 없음

app.js

express프레임워크를 사용하여 웹서버를 설정하였습니다.

서버주소-http://localhost:3000

//쿼리에서의 ?

? : 플레이스홀더(파라미터의 값에따라 사용됨)

회원가입

회원가입 화면은 위와 같으며 해당 기능에서 사용한 쿼리는 다음과 같습니다.

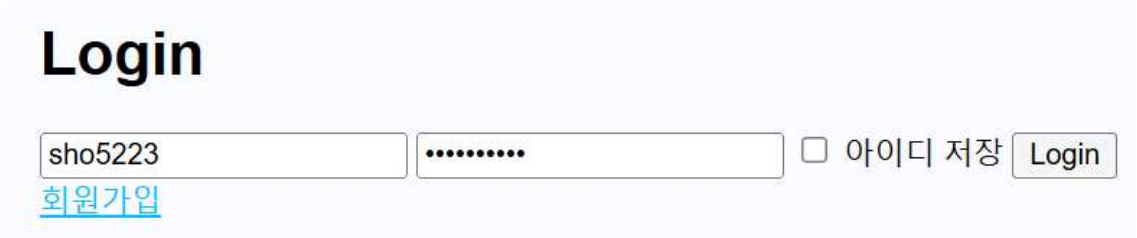
입력된 사용자 ID가 이미 데이터베이스에 존재하는지 확인합니다.
존재 시 : results[0].count가 0보다 크기 때문에 중복 여부를 알려줍니다.

```
const query = "SELECT COUNT(*) AS count FROM user WHERE username = ?";
```

회원가입 양식에서 제출한 정보를 user테이블에 그대로 삽입합니다.

```
const query = "INSERT INTO user (username, password, name, email, emaildomain) VALUES (?, ?, ?, ?, ?)";
```

로그인

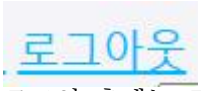


로그인 화면은 위와 같으며 해당 기능에서 사용한 쿼리는 다음과 같습니다.

```
const query = `SELECT * FROM user WHERE username = ? AND password = ?`;
```

사용자가 제출한 사용자 ID와 비밀번호로 user테이블에서 일치하는 행을 찾고 일치하는 행이 있다면 세션에 사용자 정보를 저장하고 로그인합니다.

로그아웃



로그인 후에는 로그아웃 버튼을 클릭하면 세션을 파괴하여 해당세션을 사용할 수 없게합니다.

전체 글 조회

전체 글 조회 화면은 위와 같으며 해당 기능에서 사용한 쿼리는 다음과 같습니다.

```
const sql = "SELECT postid, author, title, regdate, viewcnt, likes FROM board ORDER BY postid DESC LIMIT ?, ?";
```

board테이블에서 내용/댓글을 제외한 게시글 정보를 페이지별로 가져옵니다. 이때 페이지 번호와 페이지당 게시글 수를 기반으로 데이터베이스에서 게시글 목록을 조회합니다.**(n개단위 조회/페이징)**

아래 쿼리는 총 게시글 수를 세고 해당 게시글 수와 LIMIT를 나누어 총 페이지 수를 결정합니다.

```
const countsql = "SELECT COUNT(*) AS count FROM board";
```

게시글 상세 조회

게시글 상세 조회 화면은 위와 같으며 해당 기능에서 사용한 쿼리는 다음과 같습니다.

게시글을 클릭 할 때마다 게시글의 조회수를 1 증가시킵니다.

```
const updateviewcountsql = "UPDATE board SET viewcnt = viewcnt + 1 WHERE postid = ?";
```

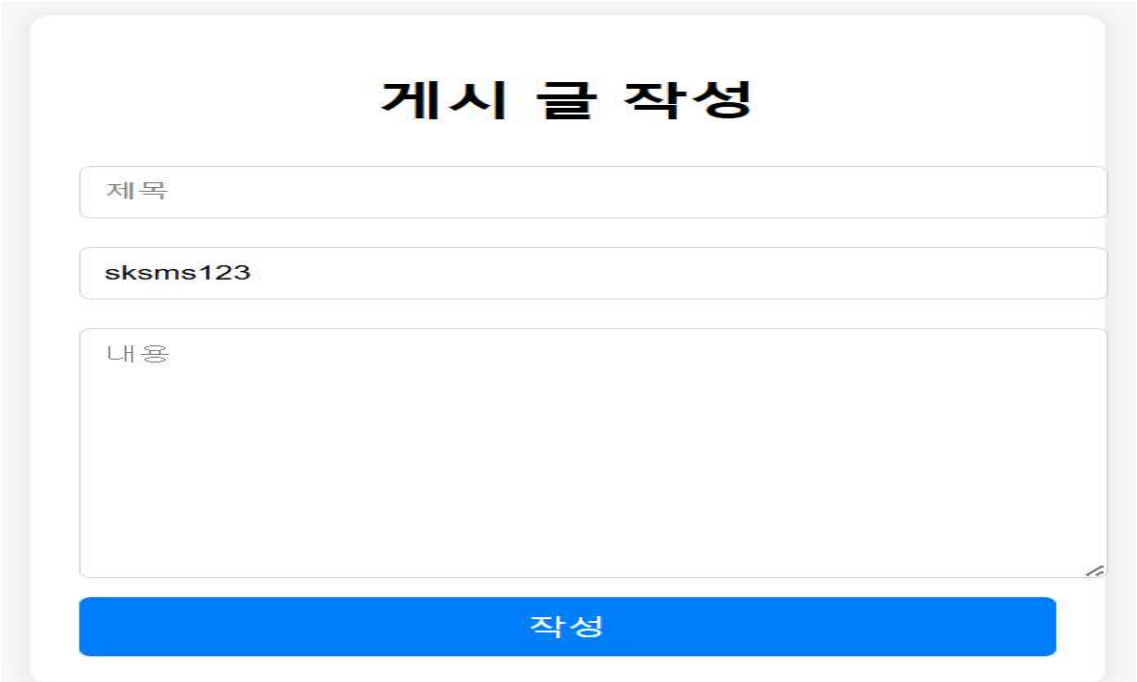
해당 게시글의 모든(*)정보를 가져옵니다.

```
const sql = "SELECT * FROM board WHERE postid=?";
```

해당 게시글에 대한 모든(*)댓글을 가져옵니다.

```
const replysql = "SELECT * FROM reply WHERE postid=?";
```

게시 글 작성

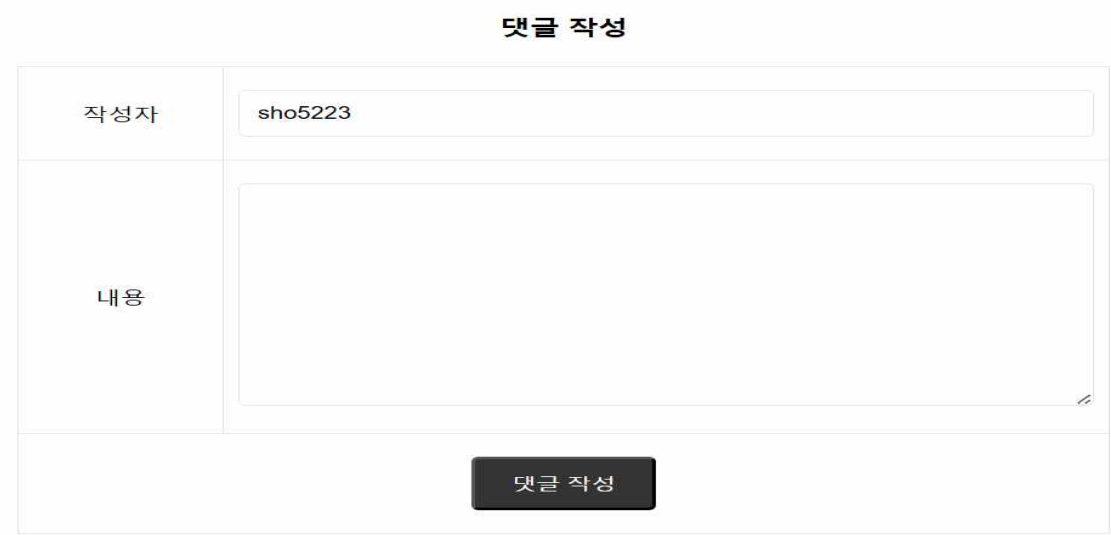


게시글 작성 화면은 위와 같으며 해당기능에서 사용한 쿼리는 다음과 같습니다.

```
const sql = "INSERT INTO board (author, title, body, regdate) VALUES (?, ?, ?, NOW())";
```

클라이언트가 작성한 게시글 정보를 받아와서 데이터베이스에 새로운 게시글로 추가합니다.

댓글작성



댓글 작성 화면은 위와 같으며 해당기능에서 사용한 쿼리는 다음과 같습니다.

클라이언트가 작성한 댓글 정보를 받아와서 해당 게시글에 대한 댓글로 데이터베이스에 추가합니다.

```
const sql = "INSERT INTO reply (postid, commentwriter, commentcontent, commentdate) VALUES (?, ?, ?, NOW())";
```

게시글 삭제

로그아웃

번호	4
제목	서현우
작성자	sho5223
작성일자	5/19/2024, 11:45:39 PM
내용	ㄹㅁㄴㄹㅁㄴㄹ
좋아요	27 좋아요
글 삭제 목록	

게시글 삭제 화면은 위와 같으며 해당 기능에서 사용한 쿼리는 다음과 같습니다.

현재 사용자 id와 writer의 정보를 비교하여 일치하면 삭제가 가능하도록 합니다.

```
const checkauthorsql = "SELECT author FROM board WHERE postid=?";
```

글삭제를 누르면 댓글이 삭제되며 이후에 게시글도 삭제됩니다.

```
connection.query("DELETE FROM reply WHERE postid=?"
```

```
connection.query("DELETE FROM board WHERE postid=?"
```

댓글 삭제

sho5223	aaaasdf	5/20/2024, 2:05:07 AM	댓글 삭제
sho5223	aaaa	5/20/2024, 4:18:03 AM	댓글 삭제

댓글 삭제 화면은 위와 같으며 해당기능에서 사용한 쿼리는 다음과 같습니다.
현재 사용자 id와 writer의 정보를 비교하여 일치하면 삭제가 가능하도록 합니다.

```
const checkauthorsql = "SELECT commentwriter, postid FROM reply WHERE commentid=?";
```

댓글 삭제를 누르면 댓글이 삭제됩니다.

```
connection.query("DELETE FROM reply WHERE commentid=?"
```

좋아요

좋아요	27 좋아요

좋아요 화면은 위와 같으며 사용한 쿼리는 다음과 같습니다.

```
const sql = "UPDATE board SET likes = likes + 1 WHERE postid = ?";
```

클라이언트가 좋아요를 요청한 게시글의 좋아요 수를 1증가시킵니다.